BACKEND DEVELOPMENT FUNDAMENTALS - YEAR II, TERM II

Lab 3

REST API, Query Parameters, Validation, and Debugging

Book API using Node.js, Express, and SQLite / Neon Postgres

Name: Vasireddy Mitesh Sai

Registration number: 252U1R1250

Section: CSE-2E-211

Faculty: Joel Prashanth D

Live deployment link:
Add the public Vercel URL here after deployment.

Repository folder: backend-development-fundamentals/lab-03

1. Aim and Overview

The aim of this lab is to build a REST API for books. The API uses normal resource names, supports searching and filtering with query parameters, checks incoming data, and returns suitable HTTP status codes.

Learning objectives

- Use HTTP methods to list, create, update, and delete books.
- Search book titles and sort, filter, and paginate the results.
- Reject invalid request data with a clear JSON message.
- Use parameterized database queries for data values.
- Practice identifying common server and database errors.

Tools and technologies

- Node.js and npm
- Express for the REST API
- SQLite for local development
- Neon Postgres for cloud deployment
- Postman for sending and checking API requests

How to run locally

Open a terminal in the `lab-03` folder and run:

```
npm install
npm run seed
npm start
```

The server normally listens at `http://localhost:3000`. To run the automated checks, use `npm test`. To restart the server automatically while editing, use `npm run dev`.

> If port 3000 is already in use, open PowerShell and run `$env:PORT=3001; npm start`, then send requests to `http://localhost:3001`.

2. API Endpoints

Method	Path	Purpose	Success status
GET	`/books`	List books, with optional search, sort, price filter, and pagination.	200
GET	`/books/:bookId`	Get one book by its ID.	200
POST	`/books`	Add a new book.	201
PUT	`/books/:bookId`	Replace the information for one book.	200
DELETE	`/books/:bookId`	Delete one book.	200

GET /books query parameters

All parameters are optional. Defaults: `order_by=id`, `order=ASC`, `limit=10`, and `offset=0`.

Parameter	Example	Meaning
`search_q`	`?search_q=the`	Find titles that contain the search text.
`order_by`	`?order_by=price`	Sort by id, title, author, price, or pages.
`order`	`?order=DESC`	Sort direction: ASC or DESC.
`limit`	`?limit=5`	Maximum number of books in the response (1-100).
`offset`	`?offset=10`	Number of matching books to skip.
`min_price`	`?min_price=8`	Only include books at or above this price.
`max_price`	`?max_price=15`	Only include books at or below this price.

Combined example:

```
GET /books?search_q=the&order_by=price&order=DESC&min_price=8&max_price=15&limit=5&offset=0
```

The search is a partial, case-insensitive match against the book title.

3. Request Validation and Responses

Example book request body

```
{
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "price": 15,
  "pages": 310
}
```

Validation rules

- `title` must be a non-empty string.
- `author` must be a non-empty string.
- `price` must be a non-negative whole number.
- `pages` must be a positive whole number.
- Sort column names and sort direction are checked before querying the database.

Status codes used

Status	When it is returned
200 OK	Books were listed, retrieved, updated, or deleted successfully.
201 Created	A book was successfully added.
400 Bad Request	The ID, query parameter, JSON, or book information is invalid.
404 Not Found	The requested book ID or route does not exist.
500 Internal Server Error	An unexpected server or database error occurred.

Error responses use a consistent JSON shape, for example:

```
{ "message": "Book not found" }
```

Database and environment settings

Locally, the API uses a SQLite file named `books.db`. For Vercel, set the secret `DATABASE_URL` environment variable to a Neon Postgres connection string. Never add the real value to source code, screenshots, or GitHub. `PORT` is optional and is usually supplied by the hosting platform.

4. Postman Test Checklist

Start the server and set the Postman base URL to `http://localhost:3000` (or the alternate port you selected). Use **Body - raw - JSON** for POST and PUT. Create a book before trying the update and delete requests.

Done	Request	Expected result
[ ]	`GET /books`	200 and a JSON array.
[ ]	`GET /books?search_q=the&order_by=price&order=DESC&min_price=8&max_price=15&limit=2&offset=0`	200 and matching sorted books.
[ ]	`GET /books/1`	200 and one book.
[ ]	`GET /books/99999`	404 and JSON message.
[ ]	`GET /books/abc`	400 and JSON message.
[ ]	`POST /books` with a valid book	201 and the new book ID.
[ ]	`POST /books` with missing or invalid fields	400 and clear JSON message.
[ ]	`PUT /books/{bookId}` with a valid book	200 and success message.
[ ]	`PUT /books/99999`	404 and JSON message.
[ ]	`DELETE /books/{bookId}`	200 and success message.
[ ]	`DELETE /books/99999`	404 and JSON message.
[ ]	`GET /books?order_by=secret`	400 and JSON message.

Automated check: run `npm test` in the lab folder.

5. Debugging Notes

Problem	How to understand and fix it
`EADDRINUSE`	The port is already used by another program. Stop that program or run the API on a different port, such as 3001.
`MODULE_NOT_FOUND`	Check the spelling of the file and its relative path. A file in the same folder is imported with `./filename`.
`no such table: book`	Check that the program uses the table named `book`. The local server creates it at startup; run `npm run seed` to add sample books.
Missing `await`	Database calls finish later. Use `await` before using their results or sending the response.
Unexpected 500 response	Read the server terminal, check the database connection, and make sure the required table exists. The API hides internal error details from clients.

Viva questions

What is a query parameter? It is an optional value after `?` in a URL that filters or changes a collection request, for example `/books?order_by=price`.

What is a path parameter? It is part of the URL path used to identify one resource, for example `/books/3`.

What does stateless mean? Each request contains the information needed to handle it; the server does not depend on conversational state from a previous request.

Which status code is used for bad input and a missing record? Use 400 for invalid input and 404 when the requested record does not exist.

Screenshot spaces

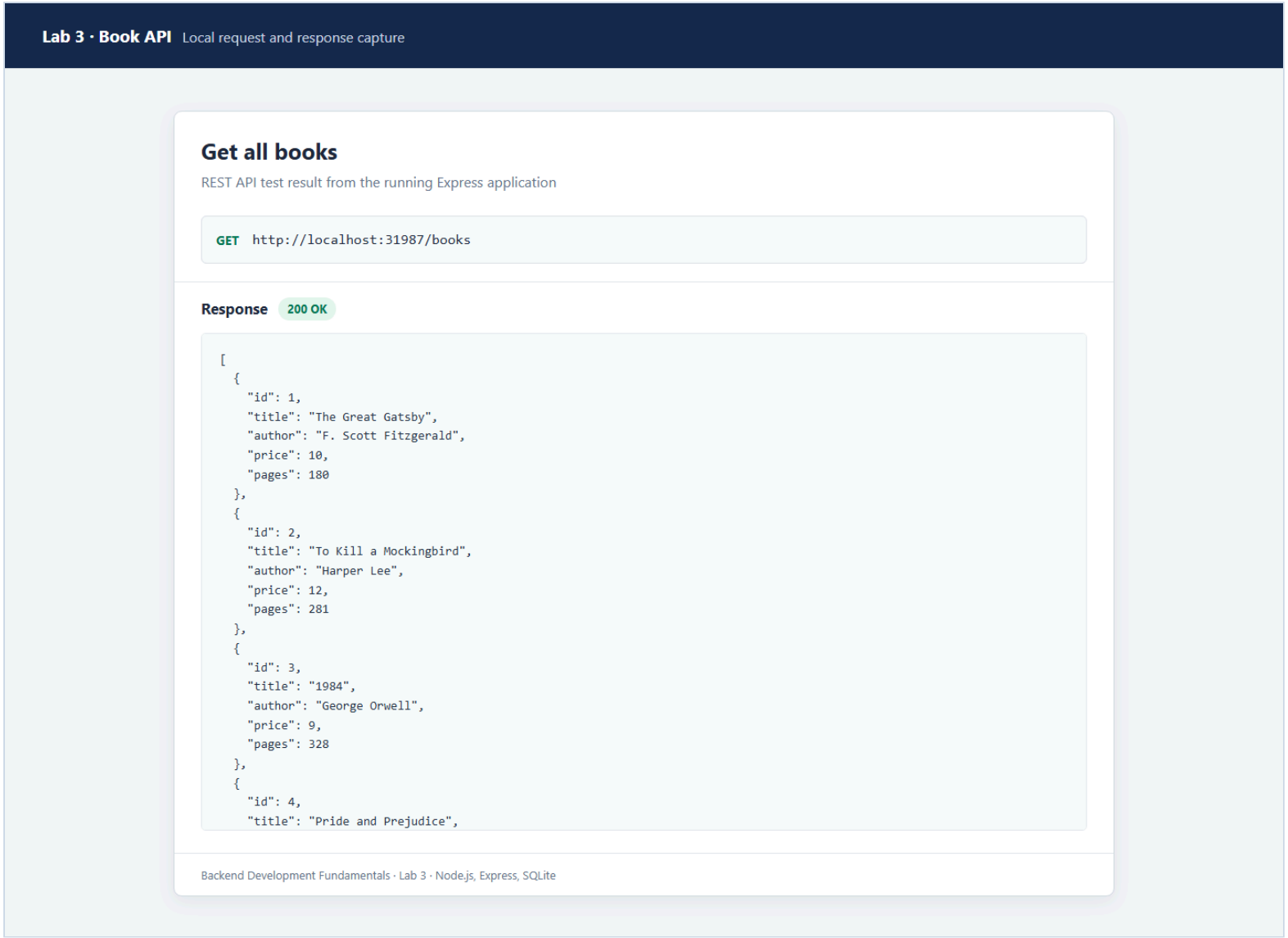
Capture your own Postman output and add the images to `lab-03/screenshots/`. Do not include passwords or database connection strings.

Screenshot to add	Paste or attach image here
GET /books, successful response	*Insert Postman screenshot here*
Filtered, sorted, paginated search	*Insert Postman screenshot here*
POST create and 201 response	*Insert Postman screenshot here*
Validation error and 400 response	*Insert Postman screenshot here*

Screenshot to add	Paste or attach image here
PUT or DELETE success, and 404 response	*Insert Postman screenshot here*

GET /books - List all books

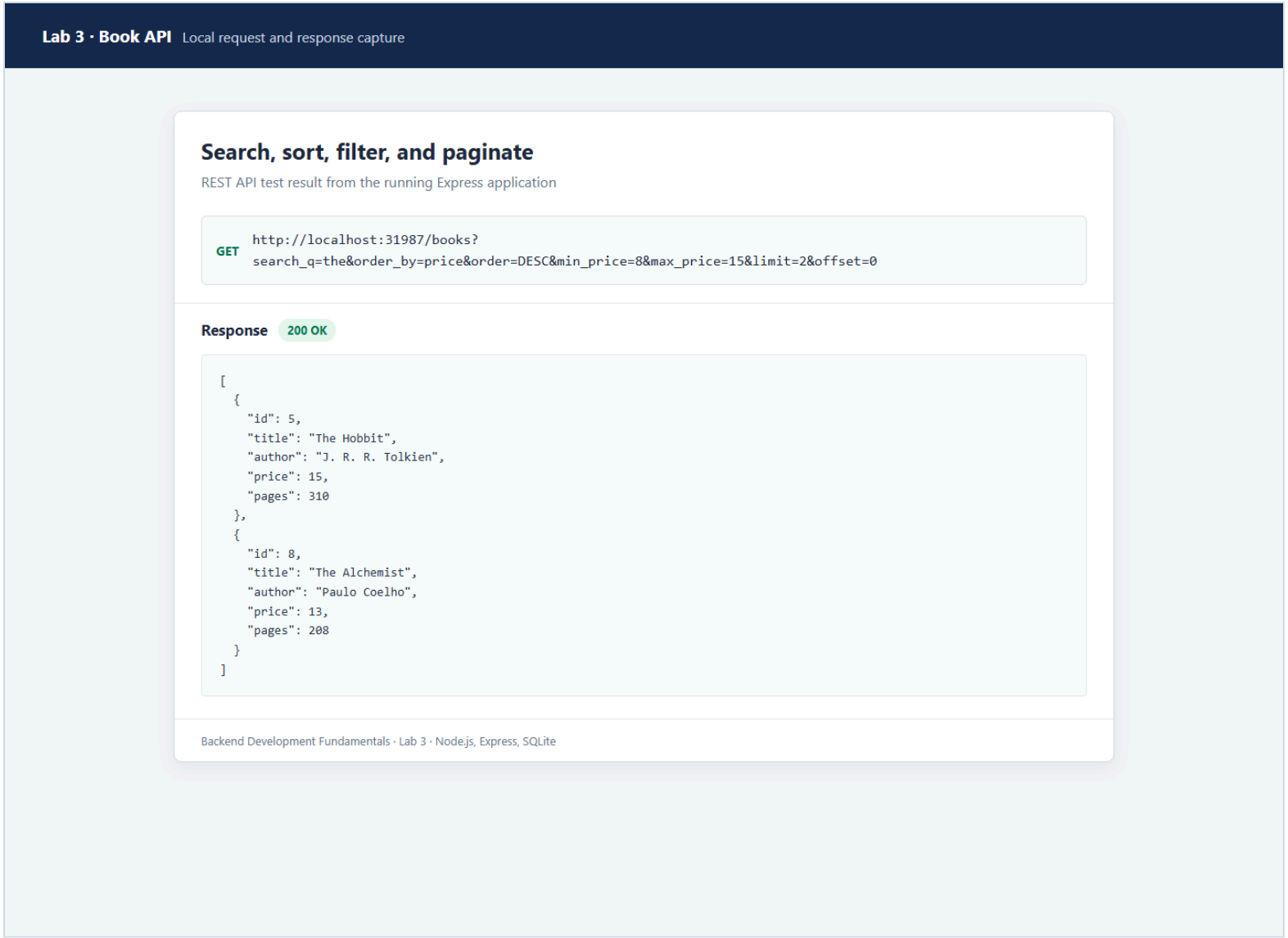
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

Search, sort, filter, and paginate

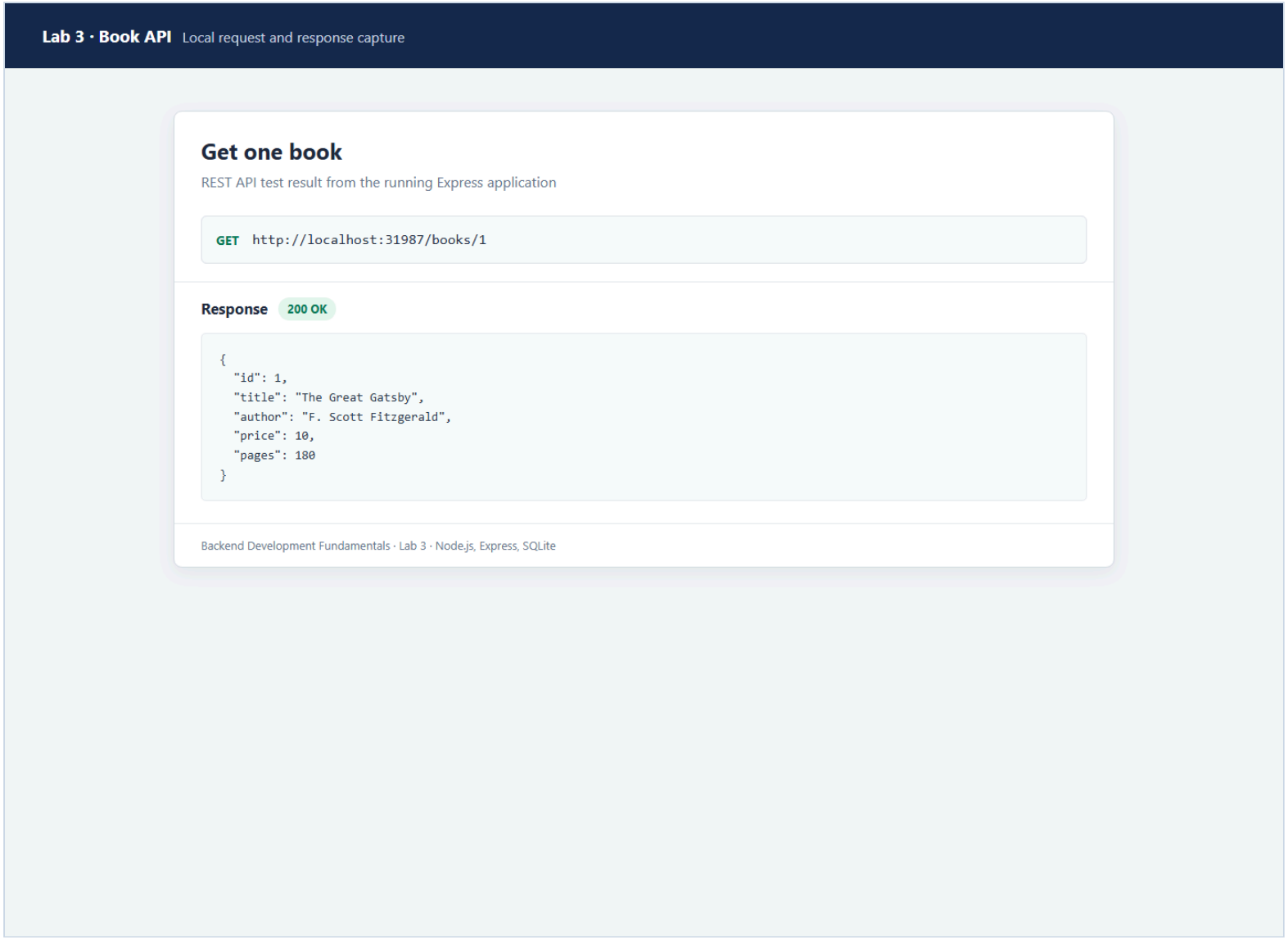
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

GET /books/1 - Get one book

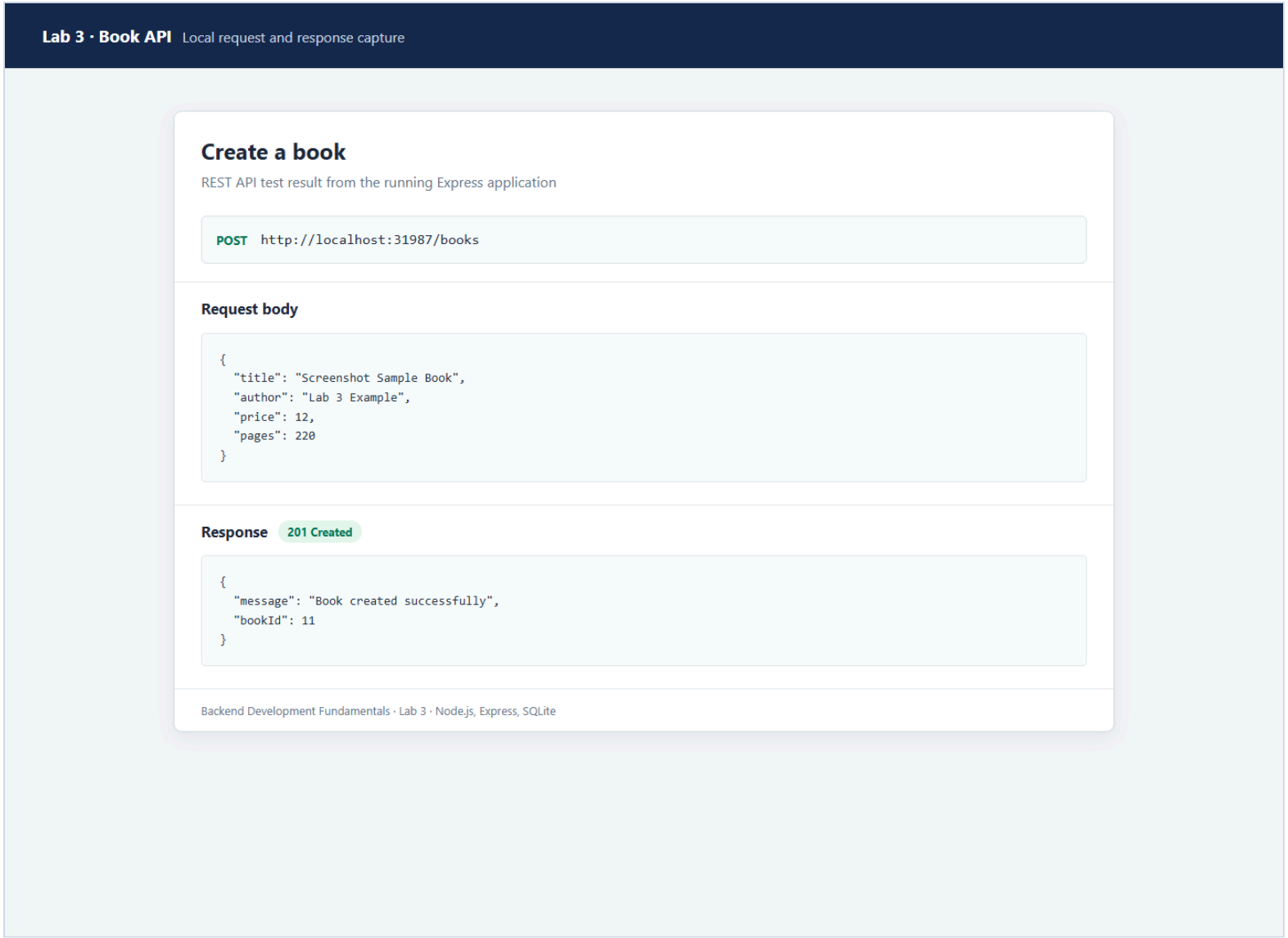
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

POST /books - Create a book

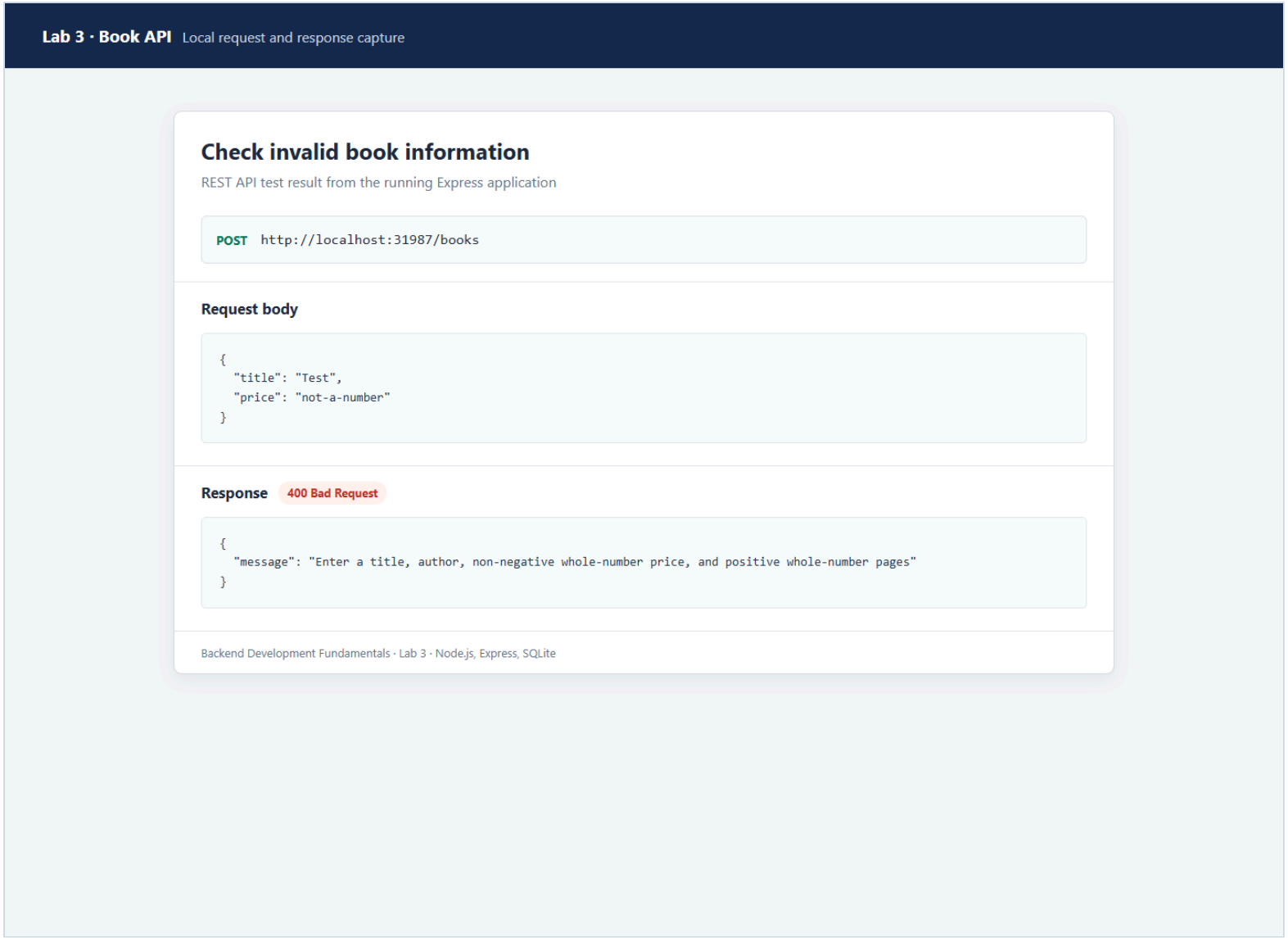
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

POST /books - Validation error

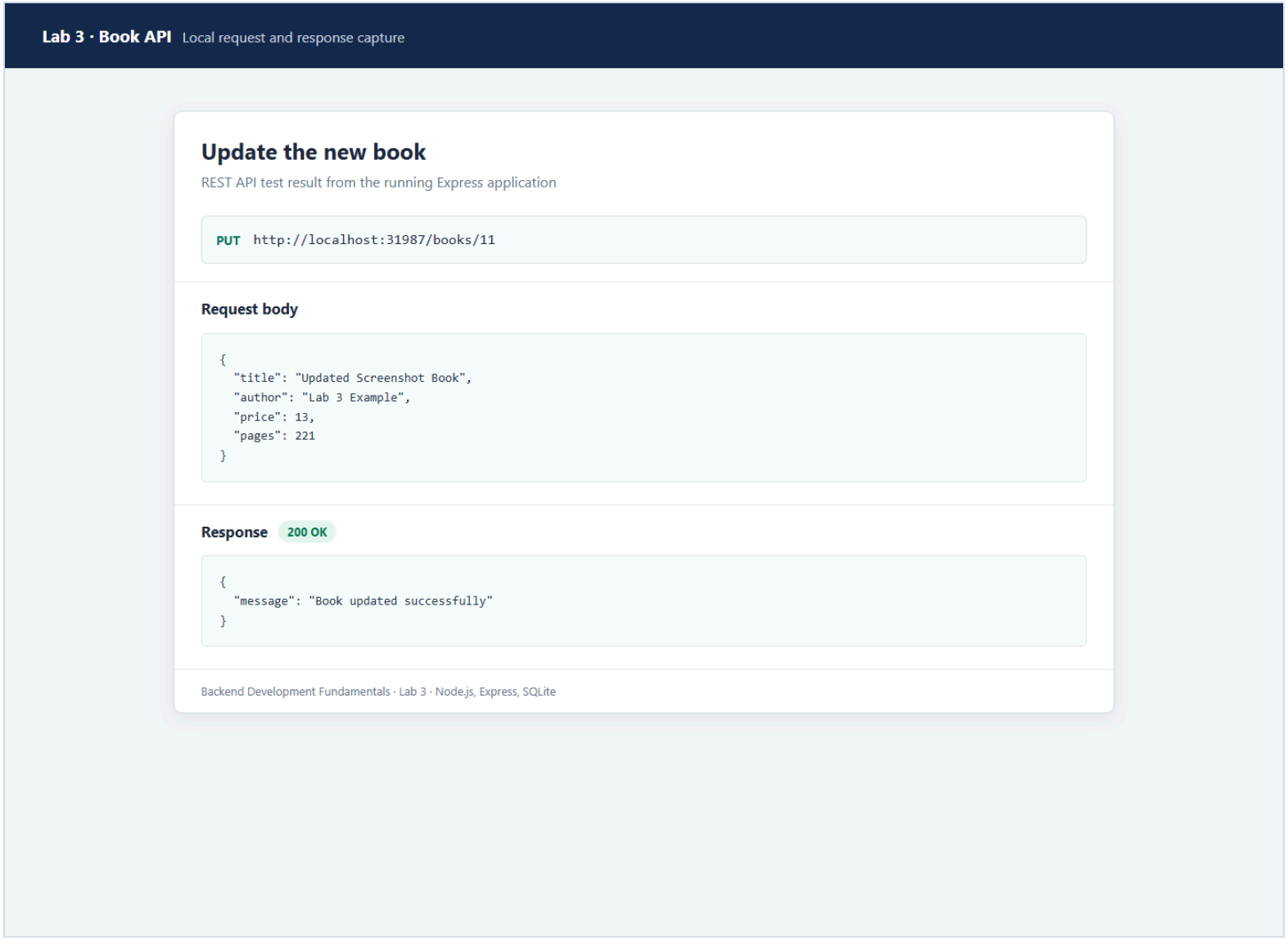
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

PUT /books/:bookId - Update a book

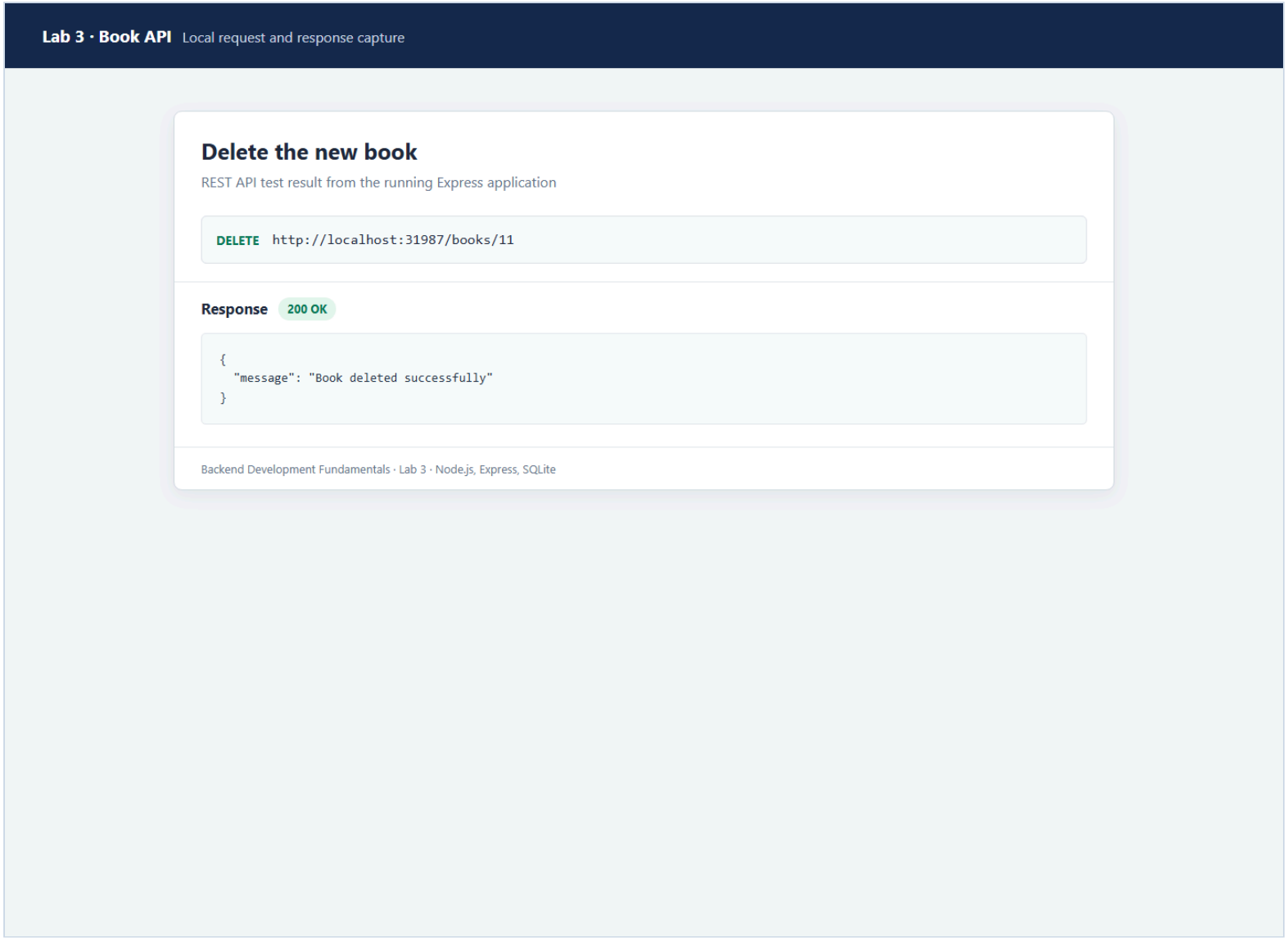
Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging

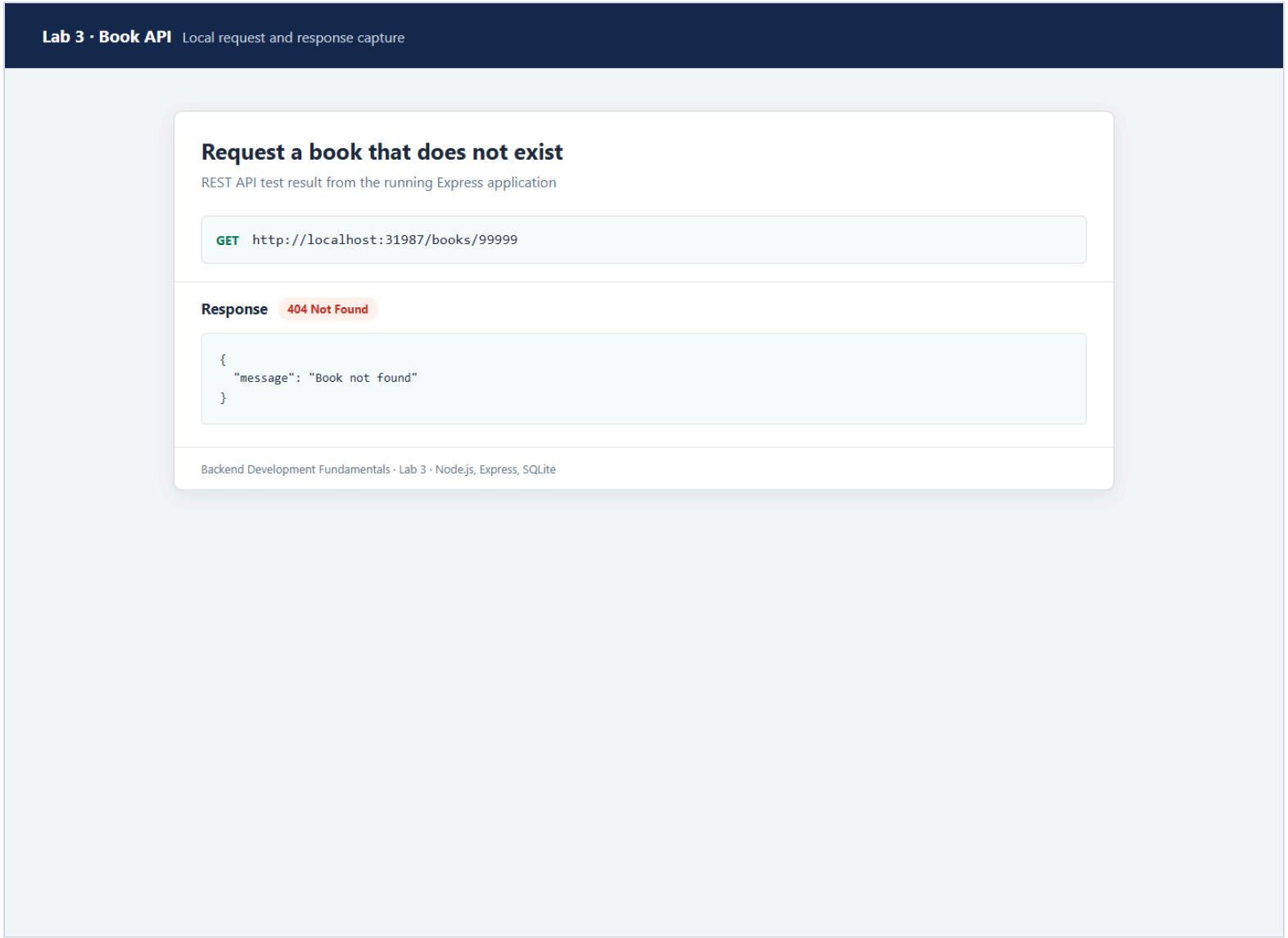
DELETE /books/:bookId - Delete a book

Actual local API response capture. This image shows the API result, not the Postman application.



GET /books/99999 - Book not found

Actual local API response capture. This image shows the API result, not the Postman application.



Lab 3: REST API, query parameters, validation, and debugging